



TSwap Protocol Audit Report

Version 1.0

Alexsander

May 26, 2026

TSwap Protocol Audit Report

Alexsander

May 26, 2026

Prepared by: Alexsander Lead Security Researcher: - Alexsander

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens
 - * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

- * [H-5] Rebase, fee-on-transfer, and ERC-777 tokens break protocol invariant
- Medium
 - * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
- Low
 - * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
- Informational
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
 - * [I-2] `PoolFactory::constructor` Lacking zero address check
 - * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`
 - * [I-4] `TSwapPool::constructor` Lacking zero address check - `wethToken` & `poolToken`
 - * [I-5] `TSwapPool` events should be indexed

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: [Uniswap Explained](#)

Disclaimer

We make all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 e643a8d4c2c802490976b538dd009b351b1c8dda
```

Scope

```
1 ./src/  
2 #-- PoolFactory.sol  
3 #-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

We definitely love auditing this codebase. We write some stateful fuzz tests to try break the protocol invariants and find some crazy weird bugs.

Issues found

Severity	Number of issues found
High	5
Medium	1
Low	2
Info	5
Total	13

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by `10_000` instead of `1_000`.

Impact: Protocol takes more fees than expected from users.

Proof of Concept:

Recommended Mitigation:

```
1     function getInputAmountBasedOnOutput(  
2         uint256 outputAmount,  
3         uint256 inputReserves,  
4         uint256 outputReserves  
5     )  
6     public  
7     pure  
8     revertIfZero(outputAmount)  
9     revertIfZero(outputReserves)  
10    returns (uint256 inputAmount)  
11    {  
12 -     return ((inputReserves * outputAmount) * 10_000) / ((  
        outputReserves - outputAmount) * 997);
```

```
13 +     return ((inputReserves * outputAmount) * 1_000) / ((
14     outputReserves - outputAmount) * 997);
    }
```

[H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The swapExactOutput function does not include any sort of slippage protection. This function is similar to what is done in TSwapPool::swapExactInput, where the function specifies a minOutputAmount, the swapExactOutput function should specify a maxInputAmount.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a swapExactOutput looking for 1 WETH a. inputToken = USDC b. outputToken = WETH c. outputAmount = 1 d. deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC

Proof Of Code

```
1     function testSlippage() public {
2         vm.startPrank(LiquidityProvider);
3         weth.approve(address(pool), 100e18);
4         poolToken.approve(address(pool), 100e18);
5         pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6         vm.stopPrank();
7
8         address whale = makeAddr("whale");
9         weth.mint(whale, 1000e18);
10        poolToken.mint(whale, 1000e18);
11        vm.startPrank(whale);
12        poolToken.approve(address(pool), 1000e18);
13        uint256 actualOutput = pool.swapExactInput(poolToken, 50e18,
14            weth, 0, uint64(block.timestamp));
15        console.log("Actual output for 50 PoolToken:", actualOutput);
16        vm.stopPrank();
17
18        console.log("Pool reserves after whale swap:");
19        console.log("WETH in pool:", weth.balanceOf(address(pool)));
20        console.log("PoolToken in pool:", poolToken.balanceOf(address(
21            pool)));
22    }
```

```
22     poolToken.approve(address(pool), type(uint256).max);
23     uint256 actualInputAmount = pool.swapExactOutput(poolToken,
24         weth, 1e18, uint64(block.timestamp));
25     console.log("Actual input amount for 1 WETH:",
26         actualInputAmount);
25     // user's swap price has doubled than expected due to the
26     // slippage caused by the whale swap
26     assert(actualInputAmount > 2e18);
27     vm.stopPrank();
28 }
```

Recommended Mitigation: We should include a maxInputAmount so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
1     function swapExactOutput(
2         IERC20 inputToken,
3     +     uint256 maxInputAmount,
4     .
5     .
6     .
7         inputAmount = getInputAmountBasedOnOutput(outputAmount,
8             inputReserves, outputReserves);
9     +     if(inputAmount > maxInputAmount){
10    +         revert();
11    +     }
11     _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The sellPoolTokens function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the poolTokenAmount parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the swapExactOutput function is called, whereas the swapExactInput function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

Recommended Mitigation: Consider changing the implementation to use swapExactInput instead of swapExactOutput. Note that this would also require changing the sellPoolTokens function to accept a new parameter (ie minWethToReceive to be passed to swapExactInput)

```
1     function sellPoolTokens(
```

```
2         uint256 poolTokenAmount,  
3 +         uint256 minWethToReceive,  
4         ) external returns (uint256 wethAmount) {  
5 -         return swapExactOutput(i_poolToken, i_wethToken,  
        poolTokenAmount, uint64(block.timestamp));  
6 +         return swapExactInput(i_poolToken, poolTokenAmount,  
        i_wethToken, minWethToReceive, uint64(block.timestamp));  
7     }
```

[H-4] In TSwapPool : : _swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: - x: The balance of the pool token - y: The balance of WETH - k: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k. However, this is broken due to the extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
1 swap_count++;  
2 if (swap_count >= SWAP_COUNT_MAX) {  
3     swap_count = 0;  
4     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);  
5 }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens 2. That user continues to swap until all the protocol funds are drained

Proof Of Code

```
1     function testInvariantBroken() public {  
2         vm.startPrank(LiquidityProvider);  
3         weth.approve(address(pool), 100e18);  
4         poolToken.approve(address(pool), 100e18);  
5         pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));  
6         vm.stopPrank();  
7  
8         uint256 outputWeth = 1e17;  
9  
10        vm.startPrank(user);  
11        poolToken.approve(address(pool), type(uint256).max);
```



```
12     poolToken.mint(user, 100e18);
13     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
14         timestamp));
15     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
16         timestamp));
17     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
18         timestamp));
19     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
20         timestamp));
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
22         timestamp));
23     int256 startingX = int256(weth.balanceOf(address(pool)));
24     int256 expectedDeltaX = int256(outputWeth) * -1;
25
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
27         timestamp));
28     vm.stopPrank();
29
30     uint256 endingX = weth.balanceOf(address(pool));
31     int256 actualDeltaX = int256(endingX) - int256(startingX);
32     assert(actualDeltaX != expectedDeltaX);
33 }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
1 -     swap_count++;
2 -     // Fee-on-transfer
3 -     if (swap_count >= SWAP_COUNT_MAX) {
4 -         swap_count = 0;
5 -         outputToken.safeTransfer(msg.sender, 1
6 -             _000_000_000_000_000_000);
7 -     }
```

[H-5] Rebase, fee-on-transfer, and ERC-777 tokens break protocol invariant

Description: Some erc20 tokens will mess with the balance, eg: rebase/fee-on-transfer tokens.

Impact: The protocol's core invariant is broken.

Proof of Concept: 1. Mock a ERC20 token which will take a fee every 5 transfer 2. Integrating the weird token into our pool 3. After 5 swaps, the protocol invariant breaks!

Proof Of Code

Put below into test/mocks/WeirdERC20.sol:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.20;
3
4 import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5
6 contract WeirdERC20 is ERC20 {
7     uint256 public constant INITIAL_SUPPLY = 1_000_000e18;
8     address public immutable owner;
9     // We take a fee once every 10 transactions
10    uint256 public count = 0;
11    uint256 public constant FEE = 10;
12    uint256 public constant USER_AMOUNT = 90;
13    uint256 public constant PRECISION = 100;
14
15    constructor() ERC20("MockYieldERC20", "MYEILD") {
16        owner = msg.sender;
17        _mint(msg.sender, INITIAL_SUPPLY);
18    }
19
20    function mint(address account, uint256 amount) external {
21        _mint(account, amount);
22    }
23
24    function burn(address account, uint256 amount) external {
25        _burn(account, amount);
26    }
27
28    function transferFrom(address from, address to, uint256 value)
29        public virtual override returns (bool) {
30        address spender = _msgSender();
31        _spendAllowance(from, spender, value);
32        if (count >= 5) {
33            uint256 userAmount = value * USER_AMOUNT / PRECISION;
34            uint256 ownerAmount = value * FEE / PRECISION;
35            count = 0;
36            _transfer(from, owner, ownerAmount);
37            _transfer(from, to, userAmount);
38        } else {
39            count++;
40            _transfer(from, to, value);
41        }
42        return true;
43    }
44 }
```

```
42     }  
43 }
```

And below snippet into TSwapPool.t.sol:

```
1     function testInvariantBrokenWithWeirdERC20Token() public {  
2         vm.startPrank(liquidityProvider);  
3         weth.approve(address(weirdTokenPool), 100e18);  
4         weirdToken.approve(address(weirdTokenPool), 100e18);  
5         weirdTokenPool.deposit(100e18, 100e18, 100e18, uint64(block.  
6             timestamp));  
7         vm.stopPrank();  
8         uint256 outputWeth = 1e17;  
9  
10        vm.startPrank(user);  
11        weirdToken.approve(address(weirdTokenPool), type(uint256).max);  
12        weirdToken.mint(user, 100e18);  
13        weirdTokenPool.swapExactOutput(weirdToken, weth, outputWeth,  
14            uint64(block.timestamp));  
14        weirdTokenPool.swapExactOutput(weirdToken, weth, outputWeth,  
15            uint64(block.timestamp));  
15        weirdTokenPool.swapExactOutput(weirdToken, weth, outputWeth,  
16            uint64(block.timestamp));  
16        weirdTokenPool.swapExactOutput(weirdToken, weth, outputWeth,  
17            uint64(block.timestamp));  
17  
18        uint256 inputTokenAmount = weirdTokenPool.  
19            getInputAmountBasedOnOutput(  
20                outputWeth, weirdToken.balanceOf(address(weirdTokenPool)),  
21                weth.balanceOf(address(weirdTokenPool))  
22            );  
21        int256 startingX = int256(weirdToken.balanceOf(address(  
22            weirdTokenPool)));  
22        int256 expectedDeltaX = int256(inputTokenAmount);  
23  
24        weirdTokenPool.swapExactOutput(weirdToken, weth, outputWeth,  
25            uint64(block.timestamp));  
25        vm.stopPrank();  
26  
27        uint256 endingX = weirdToken.balanceOf(address(weirdTokenPool))  
28            ;  
28        int256 actualDeltaX = int256(endingX) - int256(startingX);  
29  
30        assertEq(actualDeltaX, expectedDeltaX);  
31    }
```

Recommended Mitigation: Get to understand your ERC20 tokens before integrating them into your protocol. Avoid those weird tokens that will break protocol invariant.

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

```
1 Warning (5667): Unused function parameter. Remove or comment out the
   variable name to silence this warning.
2 --> src/TSwapPool.sol:96:9:
3     |
4 96  |         uint64 deadline
5     |         ^^^^^^^^^^^^^^^^^
```

Recommended Mitigation: Consider making the following change to the function:

```
1 function deposit(
2     uint256 wethToDeposit,
3     uint256 minimumLiquidityTokensToMint,
4     uint256 maximumPoolTokensToDeposit,
5     uint64 deadline
6 )
7     external
8 +     revertIfDeadlinePassed(deadline)
9     revertIfZero(wethToDeposit)
10    returns (uint256 liquidityTokensToMint)
11    {...}
```

Low

[L-1] TSwapPool::LiquidityAdded event has parameters out of order

Description: What the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```
1 - emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
2 + emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by TSwapPool::swapExactInput results in incorrect return value given

Description: The swapExactInput function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value output it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:**Recommended Mitigation:**

```
1 {
2     uint256 inputReserves = inputToken.balanceOf(address(this));
3     uint256 outputReserves = outputToken.balanceOf(address(this));
4
5 -     uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount
6 +     , inputReserves, outputReserves);
7     output = getOutputAmountBasedOnInput(inputAmount,
8     inputReserves, outputReserves);
9
10 -     if (output < minOutputAmount) {
11 -         revert TSwapPool__OutputTooLow(outputAmount,
12         minOutputAmount);
13 +     if (output < minOutputAmount) {
14 +         revert TSwapPool__OutputTooLow(outputAmount,
15         minOutputAmount);
16     }
17 }
```

Informational**[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed**

```
1 - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] PoolFactory::constructor Lacking zero address check

```
1     constructor(address wethToken) {  
2 +         if (wethToken == address(0)) {  
3 +             revert();  
4 +         }  
5         i_wethToken = wethToken;  
6     }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name()

```
1 - string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).name());  
2 + string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).symbol());
```

[I-4] TSwapPool::constructor Lacking zero address check - wethToken & poolToken

```
1     constructor(  
2         address poolToken,  
3         address wethToken,  
4         string memory liquidityTokenName,  
5         string memory liquidityTokenSymbol  
6     )  
7         ERC20(liquidityTokenName, liquidityTokenSymbol)  
8     {  
9 +         if(wethToken || poolToken == address(0)){  
10 +             revert();  
11 +         }  
12         i_wethToken = IERC20(wethToken);  
13         i_poolToken = IERC20(poolToken);  
14     }
```

[I-5] TSwapPool events should be indexed

```
1 - event Swap(address indexed swapper, IERC20 tokenIn, uint256  
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);  
2 + event Swap(address indexed swapper, IERC20 indexed tokenIn, uint256  
    amountTokenIn, IERC20 indexed tokenOut, uint256 amountTokenOut);
```